

Field Notes on Retrieval-Augmented Generation

by Ada Park and Leo Marsh

Why retrieval?

Large language models are frozen at training time and cannot see private files. Retrieval-augmented generation (RAG) fixes this by fetching relevant passages from your own documents at question time and placing them in the prompt. The model then answers from those passages instead of from memory, which sharply reduces hallucination.

How big should chunks be?

Documents are split into chunks before indexing. We recommend chunks of roughly 800 to 1,000 tokens with a small overlap, because chunks that are too small lose context while chunks that are too large dilute the embedding. Page boundaries are a reasonable first split for PDF files.

How does vector search work?

Each chunk is converted into an embedding vector and stored in a vector database such as Elasticsearch. At query time the question is embedded with the same model and a k-nearest-neighbour search returns the top five most similar chunks using cosine similarity. Filtering by conversation keeps one user's files isolated from another's.

What does memory add?

A chat window memory keeps the last ten messages of each conversation so that follow-up questions such as "and what about the second point?" can be resolved. Long-term history is stored separately in a relational database for auditing and for listing past conversations.

Why add a knowledge graph?

A knowledge graph complements vector search. Storing authors and the books they wrote in a graph database such as Neo4j lets the assistant recommend other works by the same authors, something pure similarity search cannot do.

How should we evaluate it?

The single most useful metric is groundedness: every claim in an answer should be traceable to a cited source passage. Teams should keep a small, fixed set of questions with known answers and re-run it after every change to chunking, embeddings or prompts.